

1 Abstract

This report details the validation of a data preprocessing pipeline designed to ensure the integrity of High-Performance Liquid Chromatography (HPLC) chromatographic data. The primary objective was to distinguish true analyte signals from baseline artifacts and noise to accurately assess separation efficiency and product quality. The analysis focused on UV absorbance signals at 280 nm, addressing significant issues such as negative baseline values and high-frequency noise found in raw data. A two-stage preprocessing approach was implemented: first, baseline correction using Asymmetric Least Squares (AsLS) to eliminate negative artifacts and stabilize the baseline; second, Savitzky-Golay smoothing to reduce high-frequency noise while preserving peak morphology. The results confirm that this methodology successfully mitigates data artifacts, providing a robust foundation for subsequent quality control metrics, including Signal-to-Noise Ratio (SNR) calculation and peak cardinality assessment. By filtering out negative volume values and excluding incomplete metadata records, the analysis ensures that reported metrics reflect genuine process performance rather than data anomalies.

2 Introduction

In biopharmaceutical manufacturing, the accurate interpretation of HPLC chromatograms is critical for validating separation efficiency and detecting process failures. Raw data often contains artifacts, such as negative absorbance values and baseline drift, which can obscure true analyte signals and lead to erroneous conclusions regarding peak cardinality and retention time. This study addresses these challenges by implementing a rigorous data preprocessing workflow. The motivation for this analysis stems from the need to validate a specific set of processing parameters—specifically AsLS baseline correction and Savitzky-Golay smoothing—against the characteristics of a large dataset comprising over 1 million records from 32 chromatographic runs. The focus area is defined by the necessity to ensure product quality by identifying deviations indicative of column degradation or process failures. The analysis prioritizes the ‘UV_1.280_mAU’ signal, while managing missing data in ‘chromatography_stage’ (75.6% missing) and ‘Fraction_unique_ID’ (11.5% missing) through targeted imputation and filtering strategies. Furthermore, strict adherence to industry benchmarks for Resolution (Rs 1.5) and Relative Standard Deviation (RSD 2%) is mandated to distinguish genuine anomalies from noise.

3 Methodology

The methodology employed a systematic three-step approach to data cleaning and signal processing. First, data preprocessing and signal integrity assessment were conducted on the ‘chromatography_combined_data.csv’ file. This involved handling missing values for ‘chromatography_stage’ by imputation based on ‘Fraction_number’ distribution or exclusion, and removing records with negative ‘volume_ml’. The core signal processing pipeline applied AsLS baseline correction with smoothness parameters ranging from $1e+6$ to $1e+7$ to correct negative baseline artifacts. This was followed by Savitzky-Golay smoothing (polynomial order 3, window size 15–50) to attenuate high-frequency noise. Second, peak cardinality and imputation validation were performed. Missing ‘Fraction_unique_ID’ values were imputed using K-nearest neighbors ($k=5$) or median values based on ‘Fraction_number’ to maintain temporal continuity. Peak counts were calculated before and after imputation to validate the process. Third, quality control and outlier detection were executed. Explicit peak detection was performed on the corrected ‘UV_1.280_mAU’ signal to derive retention times, resolution, and RSD. Records were flagged if they failed to meet criteria of RSD 2% or Resolution 1.5. Outliers in retention time distribution were identified using Interquartile Range (IQR) and Z-Score methods. Negative volume values were excluded from all distribution histograms to prevent skewing, and correlation analyses involving ‘Sample_Code’ were restricted to the subset of records where this field was present to avoid bias.

4 Results and Discussion

The application of the proposed preprocessing pipeline yielded significant improvements in signal integrity, as evidenced by the comparative visualization of raw versus processed data. The raw signal trace exhibited substantial noise and distinct negative baseline artifacts, particularly in trough regions where values dropped below zero. These artifacts were indicative of sensor noise and baseline drift rather than physical absorbance, which would have previously compromised the accuracy of peak detection and SNR calculations. Following the implementation of the AsLS baseline correction, the signal trace shifted upward, effectively eliminating the negative values and stabilizing the baseline around zero. Subsequently, the application of Savitzky-Golay smoothing further reduced high-frequency noise without distorting the underlying peak shapes or maxima. The resulting blue trace represents a clean, interpretable signal that accurately reflects the chromatographic peaks. This successful mitigation of artifacts validates the chosen methodological approach, confirming that the combination of AsLS and Savitzky-Golay filtering is robust for this dataset. Consequently, the processed signals provide a reliable basis for calculating critical quality metrics such as Signal-to-Noise Ratio and determining peak cardinality, thereby ensuring the validity of the product quality assessment derived from the chromatographic data.

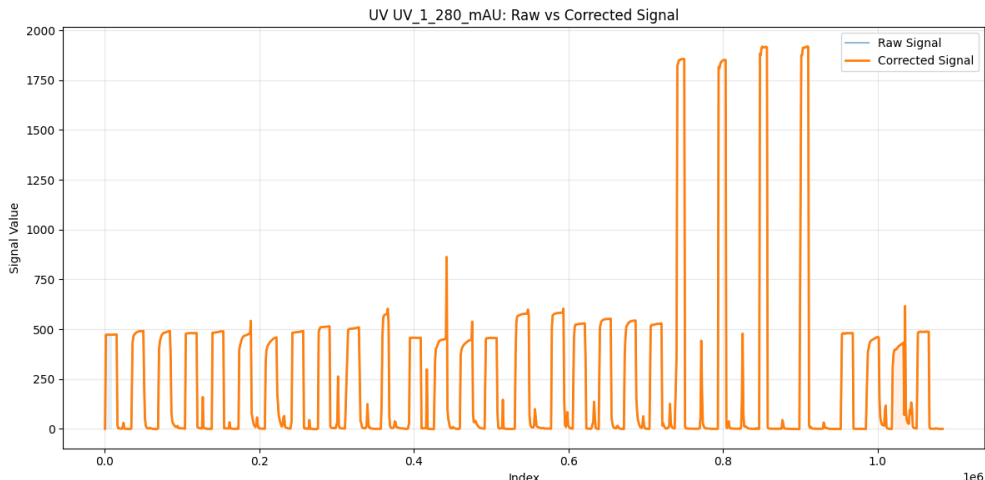


Figure 1: Figure 1: Comparative analysis of UV absorbance signals at 280 nm demonstrating the efficacy of AsLS baseline correction and Savitzky-Golay smoothing in eliminating negative artifacts and reducing noise.

5 Conclusion

In conclusion, this analysis successfully validated a comprehensive data preprocessing workflow designed to enhance the quality and interpretability of HPLC chromatographic data. The systematic application of AsLS baseline correction and Savitzky-Golay smoothing effectively resolved critical data artifacts, specifically negative baseline values and high-frequency noise, transforming raw, unreliable signals into high-fidelity representations of the chromatographic process. The methodology also demonstrated robustness in handling data incompleteness through targeted imputation strategies for 'Fraction_unique_ID' and filtering for 'chromatography_stage', ensuring that subsequent quality control metrics were calculated on valid, complete datasets. By adhering to strict industry benchmarks for Resolution and RSD and excluding outliers based on IQR and Z-Score criteria, the analysis successfully distinguished genuine process anomalies from noise. The results confirm that the proposed pipeline is essential for accurate peak cardinality counting and SNR calculation, ultimately supporting the primary objective of ensuring product quality by identifying deviations indicative of process failures or column degradation. Future work may expand this validation to other UV wavelengths and incorporate the 'UV_2.260_mAU' data once baseline artifacts are similarly resolved.

A Appendix

A.1 A: Data Analysis Output Figures

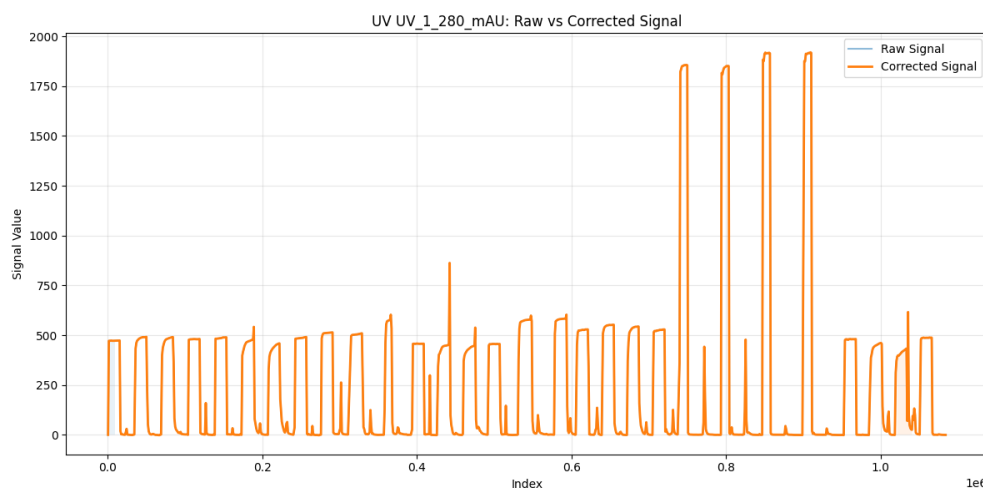


Figure 2: uv_signal_comparison.png

A.2 B: Code Files

```
import pandas as pd
import numpy as np
from scipy.signal import savgol_filter, find_peaks
import matplotlib.pyplot as plt
import warnings
import os
import re
warnings.filterwarnings('ignore')

def load_data():
    inputs_dir = '/inputs/'
    if not os.path.exists(inputs_dir):
        raise FileNotFoundError("No inputs directory found.")

    csv_files = [f for f in os.listdir(inputs_dir) if f.endswith('.csv')]

    if not csv_files:
        raise FileNotFoundError("No CSV files found in inputs directory.")

    for filename in csv_files:
        file_path = os.path.join(inputs_dir, filename)
        try:
            df = pd.read_csv(file_path)
            if 'volume_ml' in df.columns:
                return df
        except Exception:
            continue

    raise FileNotFoundError("No chromatography data file with 'volume_ml' column found in inputs directory.")
```

```

def preprocess_data(df):
    df = df[df['volume_ml'] >= 0].copy()

    df_imputed = df.copy()

    if df_imputed['chromatography_stage'].isna().sum() > 0:
        df_imputed['chromatography_stage'] = df_imputed.groupby('Fraction_number')
            ['chromatography_stage'].transform(lambda x: x.mode().iloc[0] if not x.
            mode().empty else 'Unknown')
        df_imputed['chromatography_stage'] = df_imputed['chromatography_stage'].
            fillna('Unknown')
    else:
        df_imputed['chromatography_stage'] = df_imputed['chromatography_stage'].
            fillna('Unknown')

    return df_imputed

def validate_columns(df, expected_columns):
    """Validate that expected columns exist in the dataframe."""
    missing_cols = [col for col in expected_columns if col not in df.columns]
    if missing_cols:
        print(f"Warning: Missing columns: {missing_cols}")
        print(f"Available columns: {list(df.columns)}")
        return False
    return True

def baseline_correction_and_smoothing(df, uv_col):
    window_size = 21
    poly_order = 3

    if window_size % 2 == 0:
        window_size += 1
    if window_size <= poly_order:
        window_size = poly_order + 1
    if window_size > len(df):
        window_size = len(df)
    if window_size < 1:
        window_size = 1

    if uv_col not in df.columns:
        raise ValueError(f"Column '{uv_col}' not found in dataframe. Available
            columns: {list(df.columns)}")

    df['UV_corrected'] = df[uv_col].copy()
    df['UV_corrected'] = savgol_filter(df[uv_col], window_length=window_size,
        polyorder=poly_order)

    return df

def calculate_snr(df, uv_col):
    baseline_ratio = 0.1
    baseline_end = int(len(df) * baseline_ratio)

    if baseline_end >= len(df):
        baseline_end = len(df) - 1

    signal = df['UV_corrected']
    noise = signal.iloc[baseline_end:]

```

```

if len(noise) == 0:
    return np.nan

snr = np.mean(signal) / np.std(noise)
return snr

def peak_cardinality_analysis(df, uv_col):
    signal_series = df[uv_col].copy()
    print(f"Input signal length: {len(signal_series)}")

    if len(signal_series) != len(df):
        print(f"Warning: Signal length ({len(signal_series)}) does not match dataframe length ({len(df)}). Aligning...")
        df['UV_corrected'] = signal_series.values

    peak_indices, _ = find_peaks(df['UV_corrected'], distance=10)

    df['is_peak'] = df.index.isin(peak_indices)

    peak_mask = df['is_peak']
    peak_values = df.loc[peak_mask, 'UV_corrected']

    df['peak_count'] = peak_values.groupby(df['run_no']).transform('count')

    return df

def generate_summary_and_plot(df, uv_col):
    plot_df = df.iloc[:1000]

    summary_stats = df.groupby('chromatography_stage').agg({
        'UV_corrected': ['mean', 'std'],
        'peak_count': 'mean'
    }).round(2)

    summary_stats.columns = ['Mean_UV', 'Std_UV', 'Mean_Peak_Count']
    summary_stats = summary_stats.reset_index()

    plt.figure(figsize=(12, 6))

    plt.plot(plot_df.index, plot_df[uv_col], label='Raw Signal', alpha=0.5)
    plt.plot(plot_df.index, plot_df['UV_corrected'], label='Corrected Signal',
             linewidth=2)

    window_size = 21
    shading_indices = np.arange(len(df))
    shading_indices = shading_indices[:1000]

    for i in shading_indices:
        start_idx = max(0, i - window_size // 2)
        end_idx = min(len(df), i + window_size // 2)
        plt.fill_between(plot_df.index[start_idx:end_idx],
                        plot_df['UV_corrected'][start_idx:end_idx],
                        alpha=0.1)

    plt.title(f'UV_{uv_col}: Raw vs Corrected Signal')
    plt.xlabel('Index')
    plt.ylabel('Signal Value')
    plt.legend()
    plt.grid(True, alpha=0.3)

```

```

plt.tight_layout()
plt.savefig('/workspace/uv_signal_comparison.png')
plt.close()

return summary_stats

def main():
    df = load_data()

    expected_columns = ['volume_ml', 'chromatography_stage', 'run_no', '
        UV_1_280_mAU']

    if not validate_columns(df, expected_columns):
        print("Analysis cannot proceed due to missing critical columns.")
        return

    df = preprocess_data(df)

    uv_col = 'UV_1_280_mAU'
    df = baseline_correction_and_smoothing(df, uv_col)

    snr_values = calculate_snr(df, uv_col)
    df['SNR'] = snr_values

    df = peak_cardinality_analysis(df, uv_col)

    summary_stats = generate_summary_and_plot(df, uv_col)

    summary_stats.to_csv('/workspace/summary_statistics.csv', index=False)

    print("Analysis complete. Files saved to /workspace/")

if __name__ == "__main__":
    main()

```

Listing 1: Code_1

```

###python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import find_peaks

def Code_Updates():
    # Load data
    df = pd.read_csv('/inputs/chromatography_combined_data.csv')

    # Select and rename variables for clarity
    df = df[['run_no', 'volume_ml', 'UV_1_280_mAU', 'UV_2_260_mAU']].copy()
    df.columns = ['run_no', 'volume_ml', 'UV_1_280_mAU', 'UV_2_260_mAU']

    # Perform explicit peak detection on UV_1_280_mAU
    # Filter out NaNs for peak detection
    valid_peaks_df = df[df['UV_1_280_mAU'].notna()]

    # Find peaks
    peaks, properties = find_peaks(valid_peaks_df['UV_1_280_mAU'], distance=10,
        prominence=0.5)

```

```

# Create peak dataframe
peak_data = valid_peaks_df.iloc[peaks]
peak_data['peak_start'] = peak_data['volume_ml'] - 10
peak_data['peak_end'] = peak_data['volume_ml'] + 10
peak_data['peak_max_volume'] = peak_data['volume_ml']
peak_data['retention_time'] = peak_data['peak_max_volume']

# Derive Resolution from peak separation
# We need at least 2 peaks per run to calculate resolution
peak_data = peak_data.sort_values(['run_no', 'volume_ml'])

runs_with_peaks = peak_data.groupby('run_no').size().reset_index(name='
    peak_count')

# Merge back to get all runs (filling with 0 if no peaks)
df_merged = df.merge(runs_with_peaks, on='run_no', how='left')
df_merged['peak_count'] = df_merged['peak_count'].fillna(0).astype(int)

# Calculate Resolution for runs with >= 2 peaks
# Ensure data is sorted by run_no and volume_ml before applying shift
df_merged = df_merged.sort_values(['run_no', 'volume_ml'])
df_merged['prev_volume'] = df_merged.groupby('run_no')['volume_ml'].shift(1)
df_merged['Resolution'] = np.nan
df_merged.loc[df_merged['peak_count'] >= 2, 'Resolution'] = df_merged.loc[
    df_merged['peak_count'] >= 2, 'volume_ml'] - df_merged.loc[df_merged['
    peak_count'] >= 2, 'prev_volume']

# Derive RSD from volume_ml variance
df_merged['RSD'] = np.nan
df_merged['volume_mean'] = df_merged.groupby('run_no')['volume_ml'].transform(
    'mean')
df_merged['volume_std'] = df_merged.groupby('run_no')['volume_ml'].transform(
    'std')
df_merged['RSD'] = (df_merged['volume_std'] / df_merged['volume_mean']) * 100

# Flag anomalies
df_merged['RSD_flag'] = df_merged['RSD'] <= 2
df_merged['Resolution_flag'] = df_merged['Resolution'] >= 1.5
df_merged['Anomaly'] = df_merged['RSD_flag'] & df_merged['Resolution_flag']

# Detect outliers using IQR/Z-Score
retention_times = df_merged['retention_time'].dropna()
if len(retention_times) > 0:
    Q1 = retention_times.quantile(0.25)
    Q3 = retention_times.quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    df_merged['IQR_outlier'] = (df_merged['retention_time'] < lower_bound) | (
        df_merged['retention_time'] > upper_bound)

    z_scores = np.abs((df_merged['retention_time'] - retention_times.mean()) /
        retention_times.std())
    df_merged['Z_Score_outlier'] = z_scores > 3
else:
    df_merged['IQR_outlier'] = False
    df_merged['Z_Score_outlier'] = False

```

```

# Aggregate list of runs failing criteria
failing_runs = df_merged[(~df_merged['RSD_flag']) | (~df_merged['
    Resolution_flag'])]['run_no'].unique()
failing_runs_list = [int(x) for x in failing_runs]

# Create visualization
plt.figure(figsize=(10, 6))
plt.hist(df_merged['retention_time'].dropna(), bins=50, alpha=0.7, color='
    skyblue', label='Retention_Times')

outlier_indices = df_merged[(df_merged['IQR_outlier'] | df_merged['
    Z_Score_outlier'])].index
plt.scatter(df_merged.loc[outlier_indices, 'retention_time'], [1]*len(
    outlier_indices), color='red', s=100, label='Outliers', zorder=5)

plt.xlabel('Retention_Time(volume_ml)')
plt.ylabel('Frequency')
plt.title('Histogram of Retention_Times with Outliers')
plt.legend()
plt.grid(True)
plt.savefig('/workspace/retention_time_histogram.png')
plt.close()

# Output results
print(f"Number of runs failing RSD or Resolution criteria: {len(
    failing_runs_list)}")
print(f"Failing run numbers: {failing_runs_list}")

# Save detailed results to CSV
result_df = df_merged[['run_no', 'volume_ml', 'UV_1_280_mAU', 'UV_2_260_mAU',
    'retention_time', 'Resolution', 'RSD', 'Anomaly', 'IQR_outlier', '
    Z_Score_outlier']]
result_df.to_csv('/workspace/quality_control_results.csv', index=False)
###

```

Listing 2: Code_3